

铁路图像质量感知深度识别软件源代码

软件名称：铁路图像质量感知深度识别软件 V1.0

版本号：V1.0

文档类型：源代码

目录

1. scripts/continue_overnight.sh
2. scripts/eval_faster_rcnn.py
3. scripts/eval_map.py
4. scripts/eval_ultralytics.py
5. scripts/eval.py
6. scripts/generate_sample_config.py
7. scripts/prepare_dataset.py
8. scripts/run_download_prepare.sh
9. scripts/run_formal_matrix.sh
10. scripts/run_ruod_degradation_eval.sh
11. scripts/train_faster_rcnn.py
12. scripts/train_ultralytics.py
13. scripts/train.py
14. src/__init__.py
15. src/datasets/__init__.py
16. src/datasets/underwater_detection.py
17. src/engine/__init__.py
18. src/engine/detection_ops.py
19. src/engine/trainer.py
20. src/external/__init__.py
21. src/external/faster_rcnn_runner.py
22. src/external/ultralytics_runner.py
23. src/losses/__init__.py
24. src/losses/detection_loss.py
25. src/models/__init__.py
26. src/models/modules/__init__.py
27. src/models/modules/cross_residual.py
28. src/models/modules/quality_aware_fusion.py
29. src/models/qdcr_net.py
30. src/utils/__init__.py
31. src/utils/config.py
32. src/utils/dingtalk_util.py
33. src/utils/experiment_notify.py
34. src/utils/logger.py
35. src/utils/tb_logger.py
36. src/utils/tracker.py

scripts/continue_overnight.sh

```
#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本用于夜间续跑样例实验，并在 RUOD 压缩包准备好后自动生成配置。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

GPU_UTIL_THRESHOLD="${GPU_UTIL_THRESHOLD:-20}"
GPU_MEM_USED_THRESHOLD="${GPU_MEM_USED_THRESHOLD:-2048}"
GPU_POLL_SECONDS="${GPU_POLL_SECONDS:-300}"
RUOD_POLL_SECONDS="${RUOD_POLL_SECONDS:-300}"

timestamp() {
    date '+%F %T'
}

log() {
    # 所有信息统一追加到标准输出，并由 tee 写入对应日志。
    printf '[%s] %s\n' "$(timestamp)" "$*"
}

run_logged() {
    # 包装任意命令，自动记录任务名和输出日志。
    local name="$1"
    shift
    log "running: $name"
    "$@" 2>&1 | tee "$LOG_DIR/${name}.log"
}

wait_for_gpu() {
    # 同时检查 GPU 利用率与显存占用，避免和其他任务抢卡。
    if ! command -v nvidia-smi >/dev/null 2>&1; then
        log "nvidia-smi not found, skip GPU wait"
        return 0
    fi

    while true; do
        local query
        query="$(nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv,noheader,nounits | head -n1)"
        local util
        local mem
        util="$(printf '%s' "$query" | cut -d',' -f1 | tr -d ' ')"
        mem="$(printf '%s' "$query" | cut -d',' -f2 | tr -d ' ')"

        if [[ -n "$util" && -n "$mem" && "$util" -le "$GPU_UTIL_THRESHOLD" && "$mem" -le "$GPU_MEM_USED_THRESHOLD" ]];
        then
            log "gpu is available: util=${util}% mem=${mem}MiB"
            return 0
        fi

        log "gpu busy: util=${util:-unknown}% mem=${mem:-unknown}MiB, sleep ${GPU_POLL_SECONDS}s"
        sleep "$GPU_POLL_SECONDS"
    done
}

wait_for_valid_zip() {
    # 持续轮询 RUOD 压缩包，直到文件存在且 unzip 校验通过。
    local zip_path="$1"
    while true; do
        if [[ -s "$zip_path" ]] && unzip -tqq "$zip_path" >/dev/null 2>&1; then
            log "zip ready: $zip_path"
            return 0
        fi
    done
}
```

```

    log "zip not ready: $zip_path, sleep ${RUOD_POLL_SECONDS}s"
    sleep "$RUOD_POLL_SECONDS"
done
}

prepare_ruod() {
# RUOD 数据集到位后, 自动解压并生成 QDCR/Base 两套样例配置。
local zip_path="$ROOT/data/raw/RUOD.zip"
local extract_dir="$ROOT/data/datasets/downloads/RUOD"

wait_for_valid_zip "$zip_path"
mkdir -p "$extract_dir"

if [[ ! -d "$extract_dir/train" && ! -d "$extract_dir/valid" && ! -d "$extract_dir/val" ]]; then
    run_logged "ruod_unzip" unzip -o "$zip_path" -d "$extract_dir"
else
    log "skip unzip, dataset folders already exist in $extract_dir"
fi

run_logged \
    "ruod_qdcr_config" \
    conda run -n yolo python scripts/generate_sample_config.py \
    --dataset-root "$extract_dir" \
    --template configs/qdcr_net.yaml \
    --output configs/generated/qdcr_ruod_sample.yaml \
    --experiment-name qdcr_ruod_sample \
    --epochs 3 \
    --batch-size 4 \
    --max-batches 8 \
    --image-size 320 \
    --max-objects 8

run_logged \
    "ruod_base_config" \
    conda run -n yolo python scripts/generate_sample_config.py \
    --dataset-root "$extract_dir" \
    --template configs/base.yaml \
    --output configs/generated/base_ruod_sample.yaml \
    --experiment-name baseline_ruod_sample \
    --epochs 3 \
    --batch-size 4 \
    --max-batches 8 \
    --image-size 320 \
    --max-objects 8
}

run_sample_suite() {
# 样例实验统一遵循 “训练 -> 评估” 顺序。
local config_path="$1"
local stem="$2"
wait_for_gpu
run_logged "${stem}_train" conda run -n yolo python scripts/train.py --config "$config_path"
wait_for_gpu
run_logged "${stem}_eval" conda run -n yolo python scripts/eval_map.py --config "$config_path"
}

main() {
# 先继续现有 Brackish 样例, 再在 RUOD 数据准备好后补跑 RUOD。
log "overnight continuation started"

run_sample_suite "configs/generated/qdcr_brackish_sample.yaml" "qdcr_brackish_sample"
run_sample_suite "configs/generated/base_brackish_sample.yaml" "base_brackish_sample"

prepare_ruod
run_sample_suite "configs/generated/qdcr_ruod_sample.yaml" "qdcr_ruod_sample"
run_sample_suite "configs/generated/base_ruod_sample.yaml" "base_ruod_sample"

log "overnight continuation completed"
}

```

```
main "$@"
```

scripts/eval_faster_rcnn.py

```
# -*- coding: utf-8 -*-
"""Faster R-CNN 评估入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import FasterRCNNRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate Faster R-CNN detector.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.eval_faster_rcnn")
    args = parse_args()
    config = load_config(args.config)
    runner = FasterRCNNRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f" {key}: {value:.6f}")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="评估", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_faster_rcnn] evaluation failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="评估", error=exc)
        raise

if __name__ == "__main__":
    main()
```

scripts/eval_map.py

```
# -*- coding: utf-8 -*-
"""评估并打印 mAP 指标的入口脚本。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.engine.trainer import Trainer
```

```

from src.utils import get_logger
from src.utils.config import load_config
from src.utils.experiment_notify import send_experiment_notification

def parse_args() -> argparse.Namespace:
    """解析评估参数。"""
    parser = argparse.ArgumentParser(description="Evaluate a detection checkpoint and report mAP metrics.")
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT / "configs" / "qdc_r_net.yaml",
        help="Path to the YAML config file.",
    )
    return parser.parse_args()

def main() -> None:
    """运行评估并将指标打印到标准输出。"""
    logger = get_logger("qdc_r_eval_map")
    args = parse_args()
    config = load_config(args.config)
    trainer = Trainer(config)
    start_time = datetime.now()
    try:
        metrics = trainer.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f" {key}: {value:.6f}")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=True,
            stage="评估",
            metrics=metrics,
        )
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_map] evaluation failed")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=False,
            stage="评估",
            error=exc,
        )
    raise

if __name__ == "__main__":
    main()

```

scripts/eval_ultralytics.py

```

# -*- coding: utf-8 -*-
"""Ultralytics 模型评估入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

```

```

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import UltralyticsRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Evaluate Ultralytics detectors.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.eval_ultralytics")
    args = parse_args()
    config = load_config(args.config)
    runner = UltralyticsRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.evaluate()
        end_time = datetime.now()
        print("Evaluation metrics:")
        for key, value in metrics.items():
            print(f"  {key}: {value:.6f}")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="评估", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval_ultralytics] evaluation failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="评估", error=exc)
        raise

if __name__ == "__main__":
    main()

```

scripts/eval.py

```

# -*- coding: utf-8 -*-
"""评估入口脚本。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.engine.trainer import Trainer
from src.utils import get_logger
from src.utils.config import load_config
from src.utils.experiment_notify import send_experiment_notification

def parse_args() -> argparse.Namespace:
    """解析评估命令行参数。"""
    parser = argparse.ArgumentParser(description="Evaluate a detection model.")
    parser.add_argument(
        "--config",
        type=Path,
        default=ROOT / "configs" / "qdcrc_net.yaml",
        help="Path to the YAML config file.",
    )

```

```

        return parser.parse_args()

def main() -> None:
    """加载配置并执行评估。"""
    logger = get_logger("qdc.eval")
    args = parse_args()
    config = load_config(args.config)
    trainer = Trainer(config)
    start_time = datetime.now()
    try:
        metrics = trainer.evaluate()
        end_time = datetime.now()
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=True,
            stage="评估",
            metrics=metrics,
        )
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[eval] evaluation failed")
        send_experiment_notification(
            config=config,
            project_root=ROOT,
            trainer=trainer,
            start_time=start_time,
            end_time=end_time,
            success=False,
            stage="评估",
            error=exc,
        )
    raise

if __name__ == "__main__":
    main()

```

scripts/generate_sample_config.py

```

# -*- coding: utf-8 -*-
"""根据数据集目录自动生成样例实验配置。"""

import argparse
from pathlib import Path
import sys

import yaml

ROOT = Path(__file__).resolve().parents[1]

def parse_args() -> argparse.Namespace:
    """解析配置生成脚本参数。"""
    parser = argparse.ArgumentParser(description="Generate a sample experiment config from a dataset root.")
    parser.add_argument("--dataset-root", type=Path, required=True)
    parser.add_argument("--template", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
    parser.add_argument("--experiment-name", type=str, required=True)
    parser.add_argument("--image-size", type=int, default=320)
    parser.add_argument("--epochs", type=int, default=3)
    parser.add_argument("--batch-size", type=int, default=4)
    parser.add_argument("--max-batches", type=int, default=8)
    parser.add_argument("--max-objects", type=int, default=8)

```

```

return parser.parse_args()

def find_dir(root: Path, split_names: List[str], leaf_names: set[str]) -> Path | None:
    """在数据集目录里搜索 train/val 对应的图片目录。"""
    for path in root.rglob("*"):
        if not path.is_dir():
            continue
        lower = path.as_posix().lower()
        if path.name.lower() not in leaf_names:
            continue
        if any(f"/{split}/" in lower or lower.endswith(f"/{split}") for split in split_names):
            return path
    return None

def main() -> None:
    """读取模板后按数据集结构填充实验配置。"""
    args = parse_args()
    dataset_root = args.dataset_root.resolve()
    template = yaml.safe_load(args.template.read_text(encoding="utf-8"))

    data_yaml = dataset_root / "data.yaml"
    names = template["dataset"].get("class_names", [])
    num_classes = template["dataset"].get("num_classes", len(names))
    if data_yaml.exists():
        data = yaml.safe_load(data_yaml.read_text(encoding="utf-8"))
        names = list(data.get("names", names))
        num_classes = int(data.get("nc", len(names)))

    train_images = find_dir(dataset_root, ["train"], {"images", "image"})
    val_images = find_dir(dataset_root, ["val", "valid", "validation"], {"images", "image"})
    if train_images is None:
        raise SystemExit(f"Could not find train/images under {dataset_root}")
    if val_images is None:
        # 没有单独验证集时退化为复用训练集，便于快速连通流程。
        val_images = train_images

    cfg = template
    cfg["experiment"]["name"] = args.experiment_name
    cfg["experiment"]["output_dir"] = f"outputs/checkpoints/{args.experiment_name}"
    cfg["experiment"]["runs_dir"] = f"runs/{args.experiment_name}"
    cfg["dataset"]["train_root"] = str(train_images.relative_to(ROOT))
    cfg["dataset"]["val_root"] = str(val_images.relative_to(ROOT))
    cfg["dataset"]["enhanced_root"] = str(train_images.relative_to(ROOT))
    cfg["dataset"]["num_classes"] = num_classes
    cfg["dataset"]["class_names"] = names
    cfg["dataset"]["synthetic_size"] = 0
    cfg["dataset"]["max_objects"] = args.max_objects
    cfg["dataset"]["image_size"] = args.image_size
    cfg["train"]["epochs"] = args.epochs
    cfg["train"]["batch_size"] = args.batch_size
    cfg["train"]["max_batches_per_epoch"] = args.max_batches
    cfg["eval"]["max_batches"] = args.max_batches

    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(yaml.safe_dump(cfg, sort_keys=False), encoding="utf-8")
    print(args.output)

if __name__ == "__main__":
    main()

```

scripts/prepare_dataset.py

```

# -*- coding: utf-8 -*-
"""数据集连通性检查脚本。"""

from pathlib import Path

```

```

import sys

ROOT = Path(__file__).resolve().parents[1]
# 保证直接执行脚本时也能导入项目源码。
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.datasets import UnderwaterDetectionDataset
from src.utils.config import load_config

def main() -> None:
    """读取配置并打印当前数据集索引状态。"""
    config = load_config(ROOT / "configs" / "qdc_r_net.yaml")
    dataset_config = config["dataset"]
    dataset = UnderwaterDetectionDataset(
        image_root=ROOT / dataset_config["train_root"],
        annotation_root=(ROOT / dataset_config["train_root"]).parent / "labels",
        enhanced_root=ROOT / dataset_config["enhanced_root"],
        num_classes=dataset_config["num_classes"],
        image_size=dataset_config.get("image_size", 128),
        synthetic_size=dataset_config.get("synthetic_size", 16),
        max_objects=dataset_config.get("max_objects", 8),
    )
    source = dataset[0]["source"] if len(dataset) else "empty"
    # 这里主要用于快速确认真实数据是否被发现，否则会退合成样本。
    print(f"Dataset root: {(ROOT / dataset_config['train_root']).resolve()}")
    print(f"Samples indexed: {len(dataset)}")
    print(f"Source mode: {source}")
    if len(dataset):
        sample = dataset[0]
        print(f"Objects in first sample: {int(sample['gt_classes'].numel())}")
    print("Supported real-data format: image files with YOLO labels stored as matching .txt files.")

if __name__ == "__main__":
    main()

```

scripts/run_download_prepare.sh

```

#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本负责下载公开数据集、解压，并基于模板生成样例配置。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

# 统一准备原始下载目录、解压目录、日志目录和生成配置目录。
mkdir -p data/raw data/datasets/downloads data/logs configs/generated
LOG="data/logs/download_prepare.log"

log() {
    # 所有步骤都写入同一个日志文件，便于断点排查。
    printf "[%s] %s\n" "$(date '+%F %T')" "$*" | tee -a "$LOG"
}

download_extract_generate() {
    # 针对单个数据集执行“下载 -> 解压 -> 生成 QDCR/Base 配置”的完整流水线。
    local name="$1"
    local url="$2"
    local qdc_r_cfg="$3"
    local base_cfg="$4"
    log "Downloading ${name}"
    curl -L -C - -o "data/raw/${name}.zip" "$url" >>"$LOG" 2>&1
    log "Extracting ${name}"
    unzip -o "data/raw/${name}.zip" -d "data/datasets/downloads/${name}" >>"$LOG" 2>&1
    log "Generating configs for ${name}"
    conda run -n yolo python scripts/generate_sample_config.py \

```

```

--dataset-root "data/datasets/downloads/${name}" \
--template "configs/qdcr_net.yaml" \
--output "${qdcr_cfg}" \
--experiment-name "$(basename "${qdcr_cfg}" .yaml)" >>"$LOG" 2>&1
conda run -n yolo python scripts/generate_sample_config.py \
--dataset-root "data/datasets/downloads/${name}" \
--template "configs/base.yaml" \
--output "${base_cfg}" \
--experiment-name "$(basename "${base_cfg}" .yaml)" >>"$LOG" 2>&1
log "Finished ${name}"
}

# 当前默认下载 Brackish 与 RUOD 两个公开数据集。
download_extract_generate "Brackish" "***" "configs/generated/qdcr_brackish_sample.yaml"
"configs/generated/base_brackish_sample.yaml"
download_extract_generate "RUOD" "***" "configs/generated/qdcr_ruod_sample.yaml"
"configs/generated/base_ruod_sample.yaml"

```

scripts/run_formal_matrix.sh

```

#!/usr/bin/env bash
# 使用 UTF-8 编写注释。
# 该脚本顺序执行正式实验矩阵中的训练与评估任务。
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

GPU_MEM_USED_THRESHOLD="${GPU_MEM_USED_THRESHOLD:-4500}"
GPU_POLL_SECONDS="${GPU_POLL_SECONDS:-180}"

DATASET_FILTER="${DATASET_FILTER:-}"
MODEL_FILTER="${MODEL_FILTER:-}"
CONFIG_GLOB="${CONFIG_GLOB:-configs/formal/*.yaml}"

timestamp() {
    date '+%F %T'
}

log() {
    printf "[%s] %s\n" "$(timestamp)" "$*"
}

wait_for_gpu() {
    # 如果机器支持 nvidia-smi, 则等待显存占用下降到阈值以下后再启动任务。
    if ! command -v nvidia-smi >/dev/null 2>&1; then
        return 0
    fi

    while true; do
        local used
        used="$(nvidia-smi --query-gpu=memory.used --format=csv,noheader,nounits | head -n1 | tr -d ' ')"
        if [[ -n "$used" && "$used" -le "$GPU_MEM_USED_THRESHOLD" ]]; then
            log "gpu ready: memory.used=${used}MiB"
            return 0
        fi
        log "gpu busy: memory.used=${used:-unknown}MiB, sleep ${GPU_POLL_SECONDS}s"
        sleep "$GPU_POLL_SECONDS"
    done
}

run_one() {
    # 单个配置先训练再评估, 并把日志分别落到自动化输出目录。
    local config_path="$1"
    local stem
    local model_name
}

```

```

local train_cmd
local eval_cmd
stem="$(basename "$config_path" .yaml)"
model_name="$(python - <<PY
import yaml
from pathlib import Path
cfg = yaml.safe_load(Path("$config_path").read_text())
print(str(cfg.get("model", {}).get("name", "qdcn_net"))))
PY
)"

case "$model_name" in
    yolov8|rtddetr|underwater_enhance_yolov8)
        train_cmd=(python scripts/train_ultralytics.py --config "$config_path")
        eval_cmd=(python scripts/eval_ultralytics.py --config "$config_path")
        ;;
    faster_rcnn)
        train_cmd=(python scripts/train_faster_rcnn.py --config "$config_path")
        eval_cmd=(python scripts/eval_faster_rcnn.py --config "$config_path")
        ;;
    *)
        train_cmd=(python scripts/train.py --config "$config_path")
        eval_cmd=(python scripts/eval_map.py --config "$config_path")
        ;;
esac

wait_for_gpu
log "train start: $config_path model=${model_name}"
conda run -n yolo "${train_cmd[@]}" 2>&1 | tee "$LOG_DIR/${stem}_train.log"

wait_for_gpu
log "eval start: $config_path model=${model_name}"
conda run -n yolo "${eval_cmd[@]}" 2>&1 | tee "$LOG_DIR/${stem}_eval.log"
}

should_run() {
    local config_path="$1"
    local stem
    stem="$(basename "$config_path" .yaml)"
    if [[ -n "$DATASET_FILTER" && ! "$stem" =~ $DATASET_FILTER ]]; then
        return 1
    fi
    if [[ -n "$MODEL_FILTER" && ! "$stem" =~ $MODEL_FILTER ]]; then
        return 1
    fi
    return 0
}

collect_configs() {
    local config_path
    for config_path in $CONFIG_GLOB; do
        [[ -f "$config_path" ]] || continue
        should_run "$config_path" || continue
        printf '%s\n' "$config_path"
    done | sort
}

main() {
    # 默认自动发现 configs/formal/*.yaml, 后续新增模型配置后脚本无需再次硬编码。
    # 示例:
    # MODEL_FILTER='^(base|qdcn)_' bash scripts/run_formal_matrix.sh
    # DATASET_FILTER='ruod' bash scripts/run_formal_matrix.sh
    # MODEL_FILTER='^(yolov8|rtddetr)_' DATASET_FILTER='brackish' bash scripts/run_formal_matrix.sh
    mapfile -t configs < <(collect_configs)
    if [[ "${#configs[@]}" -eq 0 ]]; then
        log "no formal configs matched; CONFIG_GLOB=${CONFIG_GLOB} DATASET_FILTER=${DATASET_FILTER:-<empty>}"
        MODEL_FILTER=${MODEL_FILTER:-<empty>}
    fi
    exit 1

    log "formal matrix runner started"
}

```

```

log "selected configs (${#configs[@]}): ${configs[*]}"
for config_path in "${configs[@]"; do
    run_one "$config_path"
done
log "formal matrix runner completed"
}

main "$@"

```

scripts/run_ruod_degradation_eval.sh

```

#!/usr/bin/env bash
set -euo pipefail

ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT"

LOG_DIR="$ROOT/outputs/automation"
mkdir -p "$LOG_DIR"

CONFIGS=(
    "configs/formal/base_ruod_testb_eval.yaml"
    "configs/formal/base_ruod_testc_eval.yaml"
    "configs/formal/base_ruod_testl_eval.yaml"
    "configs/formal/qdcr_ruod_testb_eval.yaml"
    "configs/formal/qdcr_ruod_testc_eval.yaml"
    "configs/formal/qdcr_ruod_testl_eval.yaml"
)

timestamp() {
    date '+%F %T'
}

log() {
    printf "[%s] %s\n" "$(timestamp)" "$*"
}

main() {
    log "ruod degradation evaluation started"
    for config_path in "${CONFIGS[@]"; do
        stem="$(basename "$config_path" .yaml)"
        log "eval start: $config_path"
        conda run -n yolo python scripts/eval_map.py --config "$config_path" 2>&1 | tee "$LOG_DIR/${stem}.log"
    done
    log "ruod degradation evaluation completed"
}

main "$@"

```

scripts/train_faster_rcnn.py

```

# -*- coding: utf-8 -*-
"""Faster R-CNN 训练入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import FasterRCNNRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:

```

```

parser = argparse.ArgumentParser(description="Train Faster R-CNN detector.")
parser.add_argument("--config", type=Path, required=True)
return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.train_faster_rcnn")
    args = parse_args()
    config = load_config(args.config)
    runner = FasterRCNNRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.train()
        end_time = datetime.now()
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="训练", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[train_faster_rcnn] training failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="训练", error=exc)
        raise

if __name__ == "__main__":
    main()

```

scripts/train_ultralytics.py

```

# -*- coding: utf-8 -*-
"""Ultralytics 模型训练入口。"""

import argparse
from datetime import datetime
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

from src.external import UltralyticsRunner
from src.utils import get_logger, load_config, send_experiment_notification

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Train Ultralytics detectors.")
    parser.add_argument("--config", type=Path, required=True)
    return parser.parse_args()

def main() -> None:
    logger = get_logger("qdcrc.train_ultralytics")
    args = parse_args()
    config = load_config(args.config)
    runner = UltralyticsRunner(config)
    start_time = datetime.now()
    try:
        metrics = runner.train()
        end_time = datetime.now()
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=True, stage="训练", metrics=metrics)
    except Exception as exc:
        end_time = datetime.now()
        logger.exception("[train_ultralytics] training failed")
        send_experiment_notification(config=config, project_root=ROOT, trainer=runner.notify_target(),
start_time=start_time, end_time=end_time, success=False, stage="训练", error=exc)
        raise

```

```
if __name__ == "__main__":  
    main()
```

scripts/train.py

```
# -*- coding: utf-8 -*-  
"""训练入口脚本。"""  
  
import argparse  
from datetime import datetime  
from pathlib import Path  
import sys  
  
ROOT = Path(__file__).resolve().parents[1]  
# 保证直接执行脚本时也能导入项目源码。  
if str(ROOT) not in sys.path:  
    sys.path.insert(0, str(ROOT))  
  
from src.engine.trainer import Trainer  
from src.utils import get_logger  
from src.utils.config import load_config  
from src.utils.experiment_notify import send_experiment_notification  
  
def parse_args() -> argparse.Namespace:  
    """解析训练命令行参数。"""  
    parser = argparse.ArgumentParser(description="Train a detection model.")  
    parser.add_argument(  
        "--config",  
        type=Path,  
        default=ROOT / "configs" / "qdcrc_net.yaml",  
        help="Path to the YAML config file.",  
    )  
    return parser.parse_args()  
  
def main() -> None:  
    """加载配置并启动训练流程。"""  
    logger = get_logger("qdcrc.train")  
    args = parse_args()  
    config = load_config(args.config)  
    trainer = Trainer(config)  
    start_time = datetime.now()  
    try:  
        summary = trainer.fit()  
        end_time = datetime.now()  
        send_experiment_notification(  
            config=config,  
            project_root=ROOT,  
            trainer=trainer,  
            start_time=start_time,  
            end_time=end_time,  
            success=True,  
            stage="训练",  
            summary=summary,  
        )  
    except Exception as exc:  
        end_time = datetime.now()  
        logger.exception("[train] training failed")  
        send_experiment_notification(  
            config=config,  
            project_root=ROOT,  
            trainer=trainer,  
            start_time=start_time,  
            end_time=end_time,  
            success=False,  
            stage="训练",  
            error=exc,
```

```
)
raise

if __name__ == "__main__":
    main()
```

src/__init__.py

```
# -*- coding: utf-8 -*-
"""QDCR-Net 源码主包。"""
```

src/datasets/__init__.py

```
# -*- coding: utf-8 -*-
"""数据集模块导出口。"""

from .underwater_detection import UnderwaterDetectionDataset

__all__ = ["UnderwaterDetectionDataset"]
```

src/datasets/underwater_detection.py

```
# -*- coding: utf-8 -*-
"""水下目标检测数据集定义。"""

from dataclasses import dataclass, field
import hashlib
from pathlib import Path
import tempfile
from typing import Any

from PIL import Image
import torch
from torch.utils.data import Dataset
from torchvision import transforms

@dataclass
class UnderwaterDetectionDataset(Dataset):
    """支持真实 YOLO 标注与合成回退样本的数据集。"""

    image_root: Path
    annotation_root: Path | None = None
    enhanced_root: Path | None = None
    num_classes: int = 4
    image_size: int = 128
    synthetic_size: int = 32
    max_objects: int = 8
    cache_dir: Path | None = None
    _samples: list[dict[str, Any]] = field(init=False, repr=False, default_factory=list)

    def __post_init__(self) -> None:
        """初始化路径、图像变换，并优先索引真实数据。"""
        self.image_root = Path(self.image_root)
        self.annotation_root = Path(self.annotation_root) if self.annotation_root else None
        self.enhanced_root = Path(self.enhanced_root) if self.enhanced_root else None
        self.cache_dir = Path(self.cache_dir) if self.cache_dir else None
        self.transform = transforms.Compose(
            [
                transforms.Resize((self.image_size, self.image_size)),
                transforms.ToTensor(),
            ]
        )

        image_files = self._discover_images(self.image_root)
```

```

    if image_files:
        for image_path in image_files:
            sample = self._build_real_sample(image_path)
            if sample is not None:
                self._samples.append(sample)

    if self._samples:
        return

    # 当本地没有可用真实数据时, 自动构造合成样本以保证训练链路可跑通。
    self._samples = [self._build_synthetic_sample(index) for index in range(self.synthetic_size)]

def __len__(self) -> int:
    return len(self._samples)

def __getitem__(self, index: int) -> dict[str, Any]:
    """按索引返回一个样本, 统一输出原图、增强图和标注。"""
    sample = self._samples[index]
    cached = self._load_cached_pair(sample)
    if cached is not None:
        raw_image, enhanced_image = cached
    elif sample["source"] == "real":
        raw_image, enhanced_image = self._build_real_pair(sample)
    else:
        # 合成样本直接动态生成一对原图/增强图, 避免额外落盘。
        raw_image, enhanced_image = self._build_synthetic_pair(sample["sample_id"], sample["annotations"])

    gt_boxes = torch.tensor(
        [annotation["bbox"] for annotation in sample["annotations"]],
        dtype=torch.float32,
    )
    gt_classes = torch.tensor(
        [annotation["class_id"] for annotation in sample["annotations"]],
        dtype=torch.long,
    )
    return {
        "sample_id": sample["sample_id"],
        "raw_image": raw_image,
        "enhanced_image": enhanced_image,
        "gt_boxes": gt_boxes,
        "gt_classes": gt_classes,
        "source": sample["source"],
    }

def prepare_cache(self, cache_dir: Path | None = None) -> None:
    """在训练前把图像变换结果预写到临时目录, 减少运行时 CPU 开销。"""
    target_dir = Path(cache_dir) if cache_dir is not None else self.cache_dir
    if target_dir is None:
        target_dir = Path(tempfile.gettempdir()) / "qdcrr_preprocessed" / self._cache_namespace()
    target_dir.mkdir(parents=True, exist_ok=True)
    self.cache_dir = target_dir

    for sample in self._samples:
        cache_path = self._cache_path(sample)
        if cache_path is None or cache_path.exists():
            continue
        if sample["source"] == "real":
            raw_image, enhanced_image = self._build_real_pair(sample)
        else:
            raw_image, enhanced_image = self._build_synthetic_pair(sample["sample_id"], sample["annotations"])
        torch.save(
            {
                "raw_image": raw_image.contiguous(),
                "enhanced_image": enhanced_image.contiguous(),
            },
            cache_path,
        )

def _discover_images(self, root: Path) -> List[Path]:
    """递归发现数据目录里的常见图像文件。"""
    if not root.exists():

```

```

        return []

    image_files: list[Path] = []
    for pattern in ("*.jpg", "*.jpeg", "*.png", "*.bmp"):
        image_files.extend(sorted(root.rglob(pattern)))
    return image_files

def _build_real_sample(self, image_path: Path) -> dict[str, Any] | None:
    """根据图像路径组装真实样本，如果无标注则跳过。"""
    annotations = self._load_labels_for_image(image_path)
    if not annotations:
        return None

    # 优先保留面积更大的目标，避免超出 max_objects 时随机截断。
    annotations.sort(key=lambda item: item["bbox"][2] * item["bbox"][3], reverse=True)
    return {
        "sample_id": image_path.relative_to(self.image_root).as_posix(),
        "annotations": annotations[: self.max_objects],
        "image_path": image_path,
        "enhanced_path": self._resolve_enhanced_path(image_path),
        "source": "real",
    }

def _build_synthetic_sample(self, index: int) -> dict[str, Any]:
    """构造一个可重复生成的合成样本描述。"""
    object_count = 1 + (index % min(3, self.max_objects))
    annotations = []
    for offset in range(object_count):
        label = (index + offset) % self.num_classes
        annotations.append(
            {
                "class_id": label,
                "bbox": self._synthetic_bbox(label, offset),
            }
        )
    return {
        "sample_id": f"synthetic-{index}",
        "annotations": annotations,
        "source": "synthetic",
    }

def _load_labels_for_image(self, image_path: Path) -> list[dict[str, Any]]:
    """读取 YOLO 标签文件，并过滤非法类别或越界框。"""
    label_path = self._resolve_label_path(image_path)
    if label_path is None or not label_path.exists():
        return []

    labels: list[dict[str, Any]] = []
    for raw_line in label_path.read_text(encoding="utf-8").splitlines():
        line = raw_line.strip()
        if not line:
            continue

        parts = line.split()
        if len(parts) < 5:
            continue

        try:
            class_id = int(float(parts[0]))
            bbox = [float(value) for value in parts[1:5]]
        except ValueError:
            continue

        if 0 <= class_id < self.num_classes and all(0.0 <= value <= 1.0 for value in bbox):
            labels.append({"class_id": class_id, "bbox": bbox})

    return labels

def _resolve_label_path(self, image_path: Path) -> Path | None:
    """尝试从多种常见目录结构推断标签路径。"""
    candidates: list[Path] = []

```

```

if self.annotation_root is not None:
    try:
        candidates.append(
            self.annotation_root / image_path.relative_to(self.image_root).with_suffix(".txt")
        )
    except ValueError:
        candidates.append(self.annotation_root / f"{image_path.stem}.txt")

candidates.append(image_path.with_suffix(".txt"))

parts = list(image_path.parts)
for marker in ("images", "image", "imgs", "img"):
    if marker in parts:
        marker_index = parts.index(marker)
        # 常见的 YOLO 数据集会把 images 目录平移为 labels 目录。
        remapped = Path(*parts[:marker_index], "labels", *parts[marker_index + 1 :])
        candidates.append(remapped.with_suffix(".txt"))

for candidate in candidates:
    if candidate.exists():
        return candidate
return candidates[0] if candidates else None

def _resolve_enhanced_path(self, image_path: Path) -> Path | None:
    """根据增强图根目录推断对应增强图位置。"""
    if self.enhanced_root is None or not self.enhanced_root.exists():
        return None

    candidates: list[Path] = []
    try:
        candidates.append(self.enhanced_root / image_path.relative_to(self.image_root))
    except ValueError:
        candidates.append(self.enhanced_root / image_path.name)

    for candidate in candidates:
        if candidate.exists():
            return candidate
    return None

def _load_image_tensor(self, path: Path) -> torch.Tensor:
    """读取单张图像并转换为模型输入张量。"""
    image = Image.open(path).convert("RGB")
    return self.transform(image)

def _build_real_pair(self, sample: dict[str, Any]) -> tuple[torch.Tensor, torch.Tensor]:
    raw_image = self._load_image_tensor(sample["image_path"])
    enhanced_image = (
        self._load_image_tensor(sample["enhanced_path"])
        if sample["enhanced_path"] is not None
        else self._auto_enhance(raw_image)
    )
    return raw_image, enhanced_image

def _load_cached_pair(self, sample: dict[str, Any]) -> tuple[torch.Tensor, torch.Tensor] | None:
    cache_path = self._cache_path(sample)
    if cache_path is None or not cache_path.exists():
        return None
    payload = torch.load(cache_path, map_location="cpu")
    return payload["raw_image"], payload["enhanced_image"]

def _cache_path(self, sample: dict[str, Any]) -> Path | None:
    if self.cache_dir is None:
        return None
    cache_key = hashlib.sha256(sample["sample_id"].encode("utf-8")).hexdigest()
    return self.cache_dir / f"{cache_key}.pt"

def _cache_namespace(self) -> str:
    signature = "|".join(
        [
            str(self.image_root),
            str(self.enhanced_root),

```

```

        str(self.image_size),
        str(self.max_objects),
        str(len(self._samples)),
    ]
)
return hashlib.sha256(signature.encode("utf-8")).hexdigest()[:16]

def _auto_enhance(self, image_tensor: torch.Tensor) -> torch.Tensor:
    """当缺少增强图时，使用简单亮度/对比度提升做兜底。"""
    return torch.clamp(image_tensor * 1.1 + 0.03, 0.0, 1.0)

def _build_synthetic_pair(
    self,
    sample_id: str,
    annotations: list[dict[str, Any]],
) -> tuple[torch.Tensor, torch.Tensor]:
    """根据样本 ID 稳定生成一对合成图像。"""
    seed = int(hashlib.sha256(sample_id.encode("utf-8")).hexdigest()[:8], 16)
    generator = torch.Generator().manual_seed(seed)
    raw = torch.rand((3, self.image_size, self.image_size), generator=generator) * 0.12

    for annotation in annotations:
        x1, y1, x2, y2 = self._xywh_to_xyxy(annotation["bbox"])
        left = max(int(x1 * self.image_size), 0)
        top = max(int(y1 * self.image_size), 0)
        right = min(int(x2 * self.image_size), self.image_size)
        bottom = min(int(y2 * self.image_size), self.image_size)
        channel = int(annotation["class_id"]) % 3
        # 不同类别映射到不同通道，便于让模型看到稳定的类别差异。
        raw[channel, top:bottom, left:right] += 0.75

    raw = torch.clamp(raw, 0.0, 1.0)
    enhanced = torch.clamp(raw * 1.15 + 0.05, 0.0, 1.0)
    return raw, enhanced

def _synthetic_bbox(self, label: int, slot: int) -> list[float]:
    """生成一个位于合理范围内的归一化边界框。"""
    width = max(0.14, 0.26 - 0.03 * slot)
    height = max(0.12, 0.22 - 0.02 * slot)
    center_x = min(0.18 + 0.18 * label + 0.08 * slot, 0.82)
    center_y = min(0.20 + 0.12 * label + 0.10 * slot, 0.82)
    return [center_x, center_y, width, height]

def _xywh_to_xyxy(self, bbox: list[float]) -> tuple[float, float, float, float]:
    """把中心点格式边界框转换为左上角/右下角格式。"""
    center_x, center_y, width, height = bbox
    return (
        center_x - width / 2.0,
        center_y - height / 2.0,
        center_x + width / 2.0,
        center_y + height / 2.0,
    )

```

src/engine/__init__.py

```

# -*- coding: utf-8 -*-
"""训练与检测算子模块导出口。"""

from .trainer import Trainer
from .detection_ops import compute_detection_metrics, decode_predictions, greedy_match

__all__ = ["Trainer", "compute_detection_metrics", "decode_predictions", "greedy_match"]

```

src/engine/detection_ops.py

```

# -*- coding: utf-8 -*-
"""检测任务中的匹配、解码和指标计算工具。"""

```

```

import torch
from torchvision.ops import nms

def pairwise_iou_xywh(bboxes_a: torch.Tensor, bboxes_b: torch.Tensor) -> torch.Tensor:
    """计算两组 xywh 归一化框之间的两两 IoU。"""
    if bboxes_a.numel() == 0 or bboxes_b.numel() == 0:
        return torch.zeros((bboxes_a.size(0), bboxes_b.size(0)), dtype=bboxes_a.dtype, device=bboxes_a.device)

    bboxes_b = bboxes_b.to(device=bboxes_a.device, dtype=bboxes_a.dtype)

    bboxes_a_xyxy = xywh_to_xyxy(bboxes_a)
    bboxes_b_xyxy = xywh_to_xyxy(bboxes_b)

    left_top = torch.maximum(bboxes_a_xyxy[:, None, :2], bboxes_b_xyxy[None, :, :2])
    right_bottom = torch.minimum(bboxes_a_xyxy[:, None, 2:], bboxes_b_xyxy[None, :, 2:])
    wh = (right_bottom - left_top).clamp(min=0.0)
    intersection = wh[..., 0] * wh[..., 1]

    area_a = (bboxes_a_xyxy[:, 2] - bboxes_a_xyxy[:, 0]).clamp(min=0.0) * (
        bboxes_a_xyxy[:, 3] - bboxes_a_xyxy[:, 1]
    ).clamp(min=0.0)
    area_b = (bboxes_b_xyxy[:, 2] - bboxes_b_xyxy[:, 0]).clamp(min=0.0) * (
        bboxes_b_xyxy[:, 3] - bboxes_b_xyxy[:, 1]
    ).clamp(min=0.0)
    union = area_a[:, None] + area_b[None, :] - intersection
    return intersection / union.clamp(min=1e-6)

def xywh_to_xyxy(bboxes: torch.Tensor) -> torch.Tensor:
    """把中心点格式边界框转换为角点格式。"""
    center_x = bboxes[..., 0]
    center_y = bboxes[..., 1]
    width = bboxes[..., 2]
    height = bboxes[..., 3]
    return torch.stack(
        [
            center_x - width / 2.0,
            center_y - height / 2.0,
            center_x + width / 2.0,
            center_y + height / 2.0,
        ],
        dim=-1,
    )

def greedy_match(
    logits: torch.Tensor,
    pred_boxes: torch.Tensor,
    gt_classes: torch.Tensor,
    gt_boxes: torch.Tensor,
    background_class: int,
    cls_weight: float = 1.0,
    l1_weight: float = 3.0,
    iou_weight: float = 3.0,
) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
    """基于分类、L1 与 IoU 混合代价做贪心匹配。"""
    num_queries = logits.size(0)
    matched_classes = torch.full(
        (num_queries,),
        background_class,
        dtype=torch.long,
        device=logits.device,
    )
    matched_boxes = torch.zeros((num_queries, 4), dtype=pred_boxes.dtype, device=pred_boxes.device)
    matched_mask = torch.zeros((num_queries,), dtype=torch.bool, device=logits.device)

    if gt_classes.numel() == 0:
        return matched_classes, matched_boxes, matched_mask

    probabilities = logits.softmax(dim=-1)

```

```

iou_matrix = pairwise_iou_xywh(pred_boxes, gt_boxes)
l1_matrix = torch.cdist(pred_boxes, gt_boxes, p=1)
cls_cost = -probabilities[:, gt_classes]
total_cost = cls_weight * cls_cost + l1_weight * l1_matrix + iou_weight * (1.0 - iou_matrix)

```

```

remaining_cost = total_cost.clone()
max_matches = min(num_queries, gt_classes.size(0))
inf = torch.tensor(torch.inf, device=remaining_cost.device, dtype=remaining_cost.dtype)
for _ in range(max_matches):
    flat_index = torch.argmax(remaining_cost).item()
    pred_index = flat_index // gt_classes.size(0)
    gt_index = flat_index % gt_classes.size(0)
    if not torch.isfinite(remaining_cost[pred_index, gt_index]):
        break
    matched_classes[pred_index] = gt_classes[gt_index]
    matched_boxes[pred_index] = gt_boxes[gt_index]
    matched_mask[pred_index] = True
    remaining_cost[pred_index, :] = inf
    remaining_cost[:, gt_index] = inf

return matched_classes, matched_boxes, matched_mask

```

```

def decode_predictions(
    logits: torch.Tensor,
    pred_boxes: torch.Tensor,
    background_class: int,
    conf_thresh: float,
    iou_thresh: float,
) -> list[dict[str, torch.Tensor]]:
    """把模型输出解码为按图片组织的检测结果，并做按类 NMS。"""
    batch_detections: list[dict[str, torch.Tensor]] = []
    probabilities = logits.softmax(dim=-1)
    scores, classes = probabilities[..., :-1].max(dim=-1)

    for batch_index in range(logits.size(0)):
        score = scores[batch_index]
        cls = classes[batch_index]
        box = pred_boxes[batch_index]
        keep = score >= conf_thresh
        score = score[keep]
        cls = cls[keep]
        box = box[keep]
        if score.numel() == 0:
            batch_detections.append(
                {
                    "scores": torch.zeros((0,), device=logits.device),
                    "classes": torch.zeros((0,), dtype=torch.long, device=logits.device),
                    "boxes": torch.zeros((0, 4), device=logits.device),
                }
            )
            continue

        kept_indices = []
        box_xyxy = xywh_to_xyxy(box)
        for class_id in cls.unique():
            # 按类别分别执行 NMS，避免不同类别之间相互抑制。
            class_mask = cls == class_id
            class_indices = torch.nonzero(class_mask, as_tuple=False).squeeze(1)
            selected = nms(box_xyxy[class_mask], score[class_mask], iou_thresh)
            kept_indices.append(class_indices[selected])
        final_indices = (
            torch.cat(kept_indices, dim=0)
            if kept_indices
            else torch.zeros((0,), dtype=torch.long, device=logits.device)
        )
        final_indices = final_indices[score[final_indices].argsort(descending=True)]
        batch_detections.append(
            {
                "scores": score[final_indices],
                "classes": cls[final_indices],

```

```

        "boxes": box[final_indices],
    }
)

return batch_detections

def compute_detection_metrics(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    num_classes: int,
) -> dict[str, float]:
    """计算 mAP、precision、recall 等聚合指标。"""
    ap50_per_class = []
    ap5095_per_class = []
    precisions = []
    recalls = []
    for class_id in range(num_classes):
        ap50 = _average_precision(predictions, ground_truths, class_id, 0.5)
        ap_thresholds = [
            _average_precision(predictions, ground_truths, class_id, threshold)
            for threshold in (0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95)
        ]
        precision, recall = _precision_recall(predictions, ground_truths, class_id, 0.5)
        ap50_per_class.append(ap50)
        ap5095_per_class.append(sum(ap_thresholds) / len(ap_thresholds))
        precisions.append(precision)
        recalls.append(recall)

    return {
        "map50": sum(ap50_per_class) / max(len(ap50_per_class), 1),
        "map50_95": sum(ap5095_per_class) / max(len(ap5095_per_class), 1),
        "precision": sum(precisions) / max(len(precisions), 1),
        "recall": sum(recalls) / max(len(recalls), 1),
    }

def _average_precision(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    class_id: int,
    iou_threshold: float,
) -> float:
    """针对单一类别和单一 IoU 阈值计算 AP。"""
    scored_predictions = []
    total_ground_truth = 0
    gt_lookup: dict[int, dict[str, torch.Tensor]] = {}

    for image_index, gt in enumerate(ground_truths):
        gt_mask = gt["classes"] == class_id
        gt_boxes = gt["boxes"][gt_mask]
        gt_lookup[image_index] = {
            "boxes": gt_boxes,
            "matched": torch.zeros((gt_boxes.size(0),), dtype=torch.bool),
        }
        total_ground_truth += int(gt_boxes.size(0))

    for image_index, prediction in enumerate(predictions):
        pred_mask = prediction["classes"] == class_id
        for score, box in zip(prediction["scores"][pred_mask], prediction["boxes"][pred_mask]):
            scored_predictions.append((float(score.item()), image_index, box))

    if total_ground_truth == 0:
        return 0.0

    scored_predictions.sort(key=lambda item: item[0], reverse=True)
    true_positive = []
    false_positive = []

    for _, image_index, pred_box in scored_predictions:
        gt_entry = gt_lookup[image_index]

```

```

gt_boxes = gt_entry["boxes"]
if gt_boxes.numel() == 0:
    true_positive.append(0.0)
    false_positive.append(1.0)
    continue

ious = pairwise_iou_xywh(pred_box.unsqueeze(0), gt_boxes).squeeze(0)
best_iou, best_index = torch.max(ious, dim=0)
if best_iou.item() >= iou_threshold and not gt_entry["matched"][best_index]:
    gt_entry["matched"][best_index] = True
    true_positive.append(1.0)
    false_positive.append(0.0)
else:
    true_positive.append(0.0)
    false_positive.append(1.0)

tp_cumsum = torch.tensor(true_positive).cumsum(dim=0)
fp_cumsum = torch.tensor(false_positive).cumsum(dim=0)
recalls = tp_cumsum / max(total_ground_truth, 1)
precisions = tp_cumsum / (tp_cumsum + fp_cumsum).clamp(min=1e-6)

recalls = torch.cat([torch.tensor([0.0]), recalls, torch.tensor([1.0])])
precisions = torch.cat([torch.tensor([1.0]), precisions, torch.tensor([0.0])])
# 从后向前做 precision envelope, 得到标准 PR 曲线面积。
for index in range(precisions.numel() - 2, -1, -1):
    precisions[index] = torch.maximum(precisions[index], precisions[index + 1])

recall_steps = recalls[1:] - recalls[:-1]
return float(torch.sum(recall_steps * precisions[1:]).item())

def _precision_recall(
    predictions: list[dict[str, torch.Tensor]],
    ground_truths: list[dict[str, torch.Tensor]],
    class_id: int,
    iou_threshold: float,
) -> tuple[float, float]:
    """在给定 IoU 阈值下计算单类别 precision 与 recall。"""
    scored_predictions = []
    total_ground_truth = 0
    gt_lookup: dict[int, dict[str, torch.Tensor]] = {}

    for image_index, gt in enumerate(ground_truths):
        gt_mask = gt["classes"] == class_id
        gt_boxes = gt["boxes"][gt_mask]
        gt_lookup[image_index] = {
            "boxes": gt_boxes,
            "matched": torch.zeros((gt_boxes.size(0),), dtype=torch.bool),
        }
        total_ground_truth += int(gt_boxes.size(0))

    for image_index, prediction in enumerate(predictions):
        pred_mask = prediction["classes"] == class_id
        for score, box in zip(prediction["scores"][pred_mask], prediction["boxes"][pred_mask]):
            scored_predictions.append((float(score.item()), image_index, box))

    scored_predictions.sort(key=lambda item: item[0], reverse=True)
    tp = 0
    fp = 0
    for _, image_index, pred_box in scored_predictions:
        gt_entry = gt_lookup[image_index]
        gt_boxes = gt_entry["boxes"]
        if gt_boxes.numel() == 0:
            fp += 1
            continue

        ious = pairwise_iou_xywh(pred_box.unsqueeze(0), gt_boxes).squeeze(0)
        best_iou, best_index = torch.max(ious, dim=0)
        if best_iou.item() >= iou_threshold and not gt_entry["matched"][best_index]:
            gt_entry["matched"][best_index] = True
            tp += 1

```

```

    else:
        fp += 1

precision = tp / max(tp + fp, 1)
recall = tp / max(total_ground_truth, 1)
return precision, recall

```

src/engine/trainer.py

```

# -*- coding: utf-8 -*-
"""训练器实现，负责训练、评估和实验产物写出。"""

from dataclasses import dataclass
import json
from pathlib import Path
import random
import tempfile
import time
from typing import Any

import torch
from torch import nn
from torch.utils.data import DataLoader
import yaml

from src.datasets import UnderwaterDetectionDataset
from src.engine.detection_ops import compute_detection_metrics, decode_predictions, greedy_match, pairwise_iou_xywh
from src.losses import DetectionLoss
from src.models import BaselineDetector, QDCRNet
from src.utils import ExperimentTracker, get_logger

@dataclass
class Trainer:
    """围绕配置驱动的最小训练/评估控制器。"""

    config: dict

    def __post_init__(self) -> None:
        """解析实验路径、运行设备并设置随机种子。"""
        self.logger = get_logger("qdcr.trainer")
        experiment_config = self.config.get("experiment", {})
        self.output_dir = Path(experiment_config.get("output_dir", "outputs/checkpoints/qdcr_net_smoke"))
        self.checkpoint_path = self.output_dir / "latest.pt"
        self.best_checkpoint_path = self.output_dir / "best.pt"
        self.prediction_path = self.output_dir / "predictions.json"
        self.metrics_path = self.output_dir / "metrics.json"
        self.config_snapshot_path = self.output_dir / "experiment_config.yaml"
        self.summary_path = self.output_dir / "experiment_summary.json"
        self.device = torch.device(
            experiment_config.get(
                "device",
                "cuda" if torch.cuda.is_available() else "cpu",
            )
        )
        self.run_dir: Path | None = None
        self.last_fit_summary: dict[str, Any] | None = None
        self._set_seed(int(experiment_config.get("seed", 42)))

    def fit(self) -> None:
        """执行完整训练流程，并在每轮结束后做验证与早停判断。"""
        train_dataset, val_dataset = self._build_datasets()
        model = self._build_model().to(self.device)
        loss_fn = DetectionLoss().to(self.device)
        train_config = self.config.get("train", {})
        eval_config = self.config.get("eval", {})
        early_stopping = train_config.get("early_stopping", {})
        optimizer = self._build_optimizer(model)
        tracker = ExperimentTracker(self.config)

```

```

self.run_dir = tracker.run_dir
use_amp = self.device.type == "cuda" and bool(train_config.get("amp", True))
scaler = torch.cuda.amp.GradScaler(enabled=use_amp)
num_workers = max(0, int(self.config.get("dataset", {}).get("workers", 0)))
pin_memory = self.device.type == "cuda"
persistent_workers = num_workers > 0
prefetch_factor = max(1, int(self.config.get("dataset", {}).get("prefetch_factor", 2)))
log_graph = bool(train_config.get("log_graph", False))
log_histograms = bool(train_config.get("log_histograms", False))
eval_interval = max(1, int(train_config.get("eval_interval", 5)))
batch_size = min(int(train_config.get("batch_size", 4)), len(train_dataset))
eval_batch_size = min(int(train_config.get("batch_size", 4)), len(val_dataset))
max_batches = int(train_config.get("max_batches_per_epoch", 2))
eval_max_batches = int(eval_config.get("max_batches", 2))
epochs = int(train_config.get("epochs", 1))
patience = int(early_stopping.get("patience", 5))
min_delta = float(early_stopping.get("min_delta", 1e-4))
monitor = str(early_stopping.get("monitor", "loss"))
mode = str(early_stopping.get("mode", "min")).lower()
enabled = bool(early_stopping.get("enabled", True))
log_interval = max(1, int(train_config.get("log_interval", 10)))
scalar_log_interval = max(1, int(train_config.get("scalar_log_interval", log_interval)))
# DataLoader 统一走同一个 collate_fn, 兼容每张图目标数量不同的情况。
train_loader_kwargs = {
    "batch_size": max(batch_size, 1),
    "shuffle": True,
    "num_workers": num_workers,
    "collate_fn": self._collate_batch,
    "pin_memory": pin_memory,
}
val_loader_kwargs = {
    "batch_size": max(eval_batch_size, 1),
    "shuffle": False,
    "num_workers": num_workers,
    "collate_fn": self._collate_batch,
    "pin_memory": pin_memory,
}
if num_workers > 0:
    train_loader_kwargs["persistent_workers"] = persistent_workers
    val_loader_kwargs["persistent_workers"] = persistent_workers
    train_loader_kwargs["prefetch_factor"] = prefetch_factor
    val_loader_kwargs["prefetch_factor"] = prefetch_factor
train_loader = DataLoader(
    train_dataset,
    **train_loader_kwargs,
)
val_loader = DataLoader(
    val_dataset,
    **val_loader_kwargs,
)
best_metric = float("inf") if mode == "min" else float("-inf")
epochs_without_improvement = 0
best_epoch = 0
final_val_metrics: dict[str, float] | None = None

self.logger.info(
    "[train] experiment=%s dataset_size=%s batch_size=%s epochs=%s max_batches=%s device=%s
early_stopping=%s monitor=%s mode=%s patience=%s min_delta=%.6f",
    self.config.get("experiment", {}).get("name", "unknown"),
    len(train_dataset),
    batch_size,
    epochs,
    max_batches,
    self.device,
    enabled,
    monitor,
    mode,
    patience,
    min_delta,
)
self.logger.info("[train] tensorboard_log_dir=%s", self.run_dir)

```

```

self.logger.info(
    "[train] dataloader workers=%s pin_memory=%s persistent_workers=%s prefetch_factor=%s log_interval=%s
    scalar_log_interval=%s amp=%s eval_interval=%s",
    num_workers,
    pin_memory,
    persistent_workers,
    prefetch_factor if num_workers > 0 else 0,
    log_interval,
    scalar_log_interval,
    use_amp,
    eval_interval,
)

try:
    self._write_config_snapshot()
    self._write_experiment_summary(loss_fn=loss_fn)
    self._log_hparams(tracker)
    self._prepare_dataset_cache(train_dataset, split="train")
    self._prepare_dataset_cache(val_dataset, split="val")
    if self.checkpoint_path.exists():
        try:
            # 若存在 latest checkpoint, 则优先从最近训练进度继续。
            metadata = model.load_checkpoint(
                self.checkpoint_path,
                optimizer=optimizer,
                map_location=self.device,
            )
            self.logger.info(
                "[train] resumed checkpoint=%s metadata=%s",
                self.checkpoint_path,
                metadata,
            )
        except RuntimeError as exc:
            self.logger.warning(
                "[train] skipped incompatible checkpoint=%s error=%s",
                self.checkpoint_path,
                exc,
            )

    global_step = 0
    for epoch_index in range(epochs):
        model.train()
        epoch_loss = 0.0
        epoch_box_iou = 0.0
        epoch_correct = 0
        sample_count = 0

        for batch_index, batch in enumerate(train_loader):
            if max_batches > 0 and batch_index >= max_batches:
                break

            raw_image = batch["raw_image"].to(self.device, non_blocking=pin_memory)
            enhanced_image = batch["enhanced_image"].to(self.device, non_blocking=pin_memory)
            if global_step == 0 and log_graph:
                self._try_log_graph(tracker, model, raw_image, enhanced_image)
            optimizer.zero_grad(set_to_none=True)
            with torch.amp.autocast(device_type=self.device.type, enabled=use_amp):
                predictions = model(raw_image, enhanced_image)
                # 先把固定 query 与真实目标对齐, 再进入损失计算。
                matched_classes, matched_boxes, matched_mask = self._match_batch(predictions, batch)
                loss_output = loss_fn(predictions, matched_classes, matched_boxes, matched_mask)
                loss = loss_output["loss"]
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()

            batch_iou = self._mean_iou(loss_output["pred_boxes"], matched_boxes, matched_mask)
            batch_correct = self._count_class_matches(
                loss_output["predicted_class"],
                matched_classes,
                matched_mask,

```

```

)
batch_size_value = int(matched_mask.sum().item())
batch_loss_value = float(loss.item())
batch_acc_value = batch_correct / max(batch_size_value, 1)

# 训练统计只按有效匹配目标聚合, 避免背景 query 拉低可解释性。
epoch_loss += batch_loss_value * batch_size_value
epoch_box_iou += batch_iou * batch_size_value
epoch_correct += batch_correct
sample_count += batch_size_value
global_step += 1

if global_step % scalar_log_interval == 0:
    tracker.log_scalar("train/loss_step", batch_loss_value, global_step)
    tracker.log_scalar("train/lr", self._current_lr(optimizer), global_step)
    tracker.log_scalar("train/batch_acc", batch_acc_value, global_step)
    tracker.log_scalar("train/batch_box_iou", batch_iou, global_step)
    tracker.log_scalar(
        "train/batch_cls_loss",
        float(loss_output["classification_loss"].item()),
        global_step,
    )
    tracker.log_scalar(
        "train/batch_box_loss",
        float(loss_output["box_loss"].item()),
        global_step,
    )
if (batch_index + 1) % log_interval == 0 or batch_index == 0:
    self.logger.info(
        "[train] epoch=%s batch=%s loss=%.4f acc=%.2f iou=%.2f objects=%s",
        epoch_index + 1,
        batch_index + 1,
        batch_loss_value,
        batch_acc_value,
        batch_iou,
        batch_size_value,
    )

epoch_loss_value = epoch_loss / max(sample_count, 1)
epoch_acc_value = epoch_correct / max(sample_count, 1)
epoch_iou_value = epoch_box_iou / max(sample_count, 1)
tracker.log_scalar("train/loss_epoch", epoch_loss_value, epoch_index + 1)
tracker.log_scalar("train/epoch_acc", epoch_acc_value, epoch_index + 1)
tracker.log_scalar("train/epoch_box_iou", epoch_iou_value, epoch_index + 1)
if log_histograms:
    tracker.log_parameter_histograms(model.named_parameters(), epoch_index + 1)
# 每轮先保存 latest, 便于异常中断后继续训练。
model.save_checkpoint(
    self.checkpoint_path,
    optimizer=optimizer,
    metadata={
        "experiment": self.config.get("experiment", {}).get("name", "unknown"),
        "epoch": epoch_index + 1,
        "samples": sample_count,
        "device": str(self.device),
    },
)
)
should_eval = (epoch_index + 1) % eval_interval == 0 or (epoch_index + 1) == epochs
if not should_eval:
    self.logger.info(
        "[train] epoch=%s skipped validation eval_interval=%s",
        epoch_index + 1,
        eval_interval,
    )
    continue
val_metrics = self._evaluate_model(
    model=model,
    loss_fn=loss_fn,
    val_loader=val_loader,
    max_batches=eval_max_batches,
    conf_thresh=float(eval_config.get("conf_thresh", 0.25)),

```

```

        iou_thresh=float(eval_config.get("iou_thresh", 0.5)),
        tracker=tracker,
        tracker_step=epoch_index + 1,
        metric_prefix="val",
        use_amp=use_amp,
        pin_memory=pin_memory,
    )
    final_val_metrics = val_metrics
    monitored_value = float(val_metrics[monitor])
    tracker.log_scalar(f"train/early_stop_{monitor}", monitored_value, epoch_index + 1)
    tracker.flush()
    improved = self._is_improved(
        current=monitored_value,
        best=best_metric,
        mode=mode,
        min_delta=min_delta,
    )
    if improved:
        best_metric = monitored_value
        best_epoch = epoch_index + 1
        epochs_without_improvement = 0
        # 只有监控指标改善时才刷新 best checkpoint.
        model.save_checkpoint(
            self.best_checkpoint_path,
            optimizer=optimizer,
            metadata={
                "experiment": self.config.get("experiment", {}).get("name", "unknown"),
                "epoch": epoch_index + 1,
                "samples": sample_count,
                "device": str(self.device),
                "best_metric": best_metric,
                "monitor": monitor,
            },
        )
    else:
        epochs_without_improvement += 1
    self.logger.info(
        "[train] epoch=%s summary loss=%.4f acc=%.2f iou=%.2f val_%.4f best=%.4f best_epoch=%s"
        wait=%s/%s samples=%s",
        epoch_index + 1,
        epoch_loss_value,
        epoch_acc_value,
        epoch_iou_value,
        monitor,
        monitored_value,
        best_metric,
        best_epoch,
        epochs_without_improvement,
        patience,
        sample_count,
    )
    if enabled and epochs_without_improvement >= patience:
        self.logger.info(
            "[train] early stopping triggered at epoch=%s best_epoch=%s best_%.4f",
            epoch_index + 1,
            best_epoch,
            monitor,
            best_metric,
        )
        break
    if final_val_metrics is not None:
        self._log_hparams(tracker, final_val_metrics, prefix="val")
        self.last_fit_summary = {
            "best_epoch": best_epoch,
            "best_metric": best_metric,
            "monitor": monitor,
            "last_val_metrics": final_val_metrics,
            "run_dir": str(self.run_dir) if self.run_dir is not None else None,
            "checkpoint_path": str(self.checkpoint_path),
            "best_checkpoint_path": str(self.best_checkpoint_path),
        }
}

```

```

        return self.last_fit_summary
    finally:
        tracker.close()

def evaluate(self) -> dict[str, float]:
    """加载 checkpoint 并输出评估指标、复杂度和推理速度。"""
    _, val_dataset = self._build_datasets()
    model = self._build_model().to(self.device)
    loss_fn = DetectionLoss().to(self.device)
    tracker = ExperimentTracker(self.config)
    self.run_dir = tracker.run_dir
    use_amp = self.device.type == "cuda" and bool(self.config.get("train", {}).get("amp", True))
    batch_size = min(int(self.config.get("train", {}).get("batch_size", 4)), len(val_dataset))
    max_batches = int(self.config.get("eval", {}).get("max_batches", 2))
    self._prepare_dataset_cache(val_dataset, split="eval")
    val_loader = DataLoader(
        val_dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=max(0, int(self.config.get("dataset", {}).get("workers", 0))),
        collate_fn=self._collate_batch,
        pin_memory=self.device.type == "cuda",
    )

    try:
        eval_checkpoint = self._resolve_eval_checkpoint()
        if eval_checkpoint is not None:
            try:
                # 默认优先评估 best checkpoint, 没有时回退到 latest。
                metadata = model.load_checkpoint(eval_checkpoint, map_location=self.device)
                self.logger.info("[eval] loaded checkpoint=%s metadata=%s", eval_checkpoint, metadata)
            except RuntimeError as exc:
                self.logger.warning(
                    "[eval] skipped incompatible checkpoint=%s error=%s",
                    eval_checkpoint,
                    exc,
                )
        else:
            self.logger.warning("[eval] checkpoint not found, using fresh model=%s", self.checkpoint_path)

        self.logger.info(
            "[eval] experiment=%s dataset_size=%s batch_size=%s max_batches=%s device=%s",
            self.config.get("experiment", {}).get("name", "unknown"),
            len(val_dataset),
            batch_size,
            max_batches,
            self.device,
        )

        self.logger.info("[eval] tensorboard_log_dir=%s", self.run_dir)

        model.eval()
        eval_config = self.config.get("eval", {})
        metrics = self._evaluate_model(
            model=model,
            loss_fn=loss_fn,
            val_loader=val_loader,
            max_batches=max_batches,
            conf_thresh=float(eval_config.get("conf_thresh", 0.25)),
            iou_thresh=float(eval_config.get("iou_thresh", 0.5)),
            tracker=tracker,
            tracker_step=1,
            metric_prefix="eval",
            use_amp=use_amp,
            pin_memory=self.device.type == "cuda",
        )

        # 额外补充参数量、GFLOPs 与 FPS, 便于和论文表格对齐。
        complexity_metrics = self._complexity_metrics(model, self._last_profile_batch)
        speed_metrics = self._speed_metrics(
            model=model,
            dataset=val_dataset,
            conf_thresh=float(eval_config.get("conf_thresh", 0.25)),

```

```

def add_scalar(self, tag: str, value: float, step: int) -> None:
    if self.writer is None:
        return
    self.writer.add_scalar(tag, value, step)

def flush(self) -> None:
    if self.writer is not None:
        self.writer.flush()

def close(self) -> None:
    if self.writer is not None:
        self.writer.flush()
        self.writer.close()

def _serialize(self, value: Any) -> str | bool | int | float:
    if isinstance(value, (bool, int, float, str)):
        return value
    if value is None:
        return "none"
    return str(value)

```

src/utils/tracker.py

```

# -*- coding: utf-8 -*-
"""实验跟踪工具，兼容 TSV 与 TensorBoard 两种记录方式。"""

from collections.abc import Iterable
from pathlib import Path
from typing import Any

from .tb_logger import TensorBoardLogger

class ExperimentTracker:
    """负责把训练/评估标量写入磁盘，并在可用时同步到 TensorBoard。"""

    def __init__(self, config: dict[str, Any], run_dir: Path | None = None) -> None:
        self.tb_logger = TensorBoardLogger(config)
        self.run_dir = run_dir or self.tb_logger.run_dir
        self.run_dir.mkdir(parents=True, exist_ok=True)
        self.scalar_path = self.run_dir / "scalars.tsv"

        if not self.scalar_path.exists():
            self.scalar_path.write_text("step\ttag\tvalue\n", encoding="utf-8")

    def log_scalar(self, tag: str, value: float, step: int) -> None:
        """记录单个标量，便于后续画曲线或做回归比较。"""
        with self.scalar_path.open("a", encoding="utf-8") as handle:
            handle.write(f"{step}\t{tag}\t{value:.8f}\n")

        self.tb_logger.add_scalar(tag, value, step)

    def log_hparams(self, hparams: dict[str, Any], metrics: dict[str, float] | None = None) -> None:
        """记录超参数，便于 TensorBoard HParams 面板比较实验。"""
        self.tb_logger.log_hparams(hparams, metrics)

    def log_model_graph(self, model: Any, input_to_model: Any) -> None:
        """在图可追踪时记录模型结构。"""
        self.tb_logger.add_graph(model, input_to_model)

    def log_parameter_histograms(self, named_parameters: Iterable[tuple[str, Any]], step: int, limit: int = 4) -> None:
        """记录少量关键参数的分布，避免 event 文件过大。"""
        logged = 0
        for name, parameter in named_parameters:
            if logged >= limit:
                break
            if parameter is None or not getattr(parameter, "requires_grad", False):
                continue

```

```

        if getattr(parameter, "ndim", 0) < 2:
            continue
        self.tb_logger.add_histogram(f"params/{name}", parameter.detach().cpu(), step)
        logged += 1

    def flush(self) -> None:
        """主动刷盘，减少异常退出时的日志丢失。"""
        self.tb_logger.flush()

    def close(self) -> None:
        """关闭底层 writer，确保缓存内容落盘。"""
        self.tb_logger.close()

```

configs/base_map_tuned.yaml

```

experiment:
  name: baseline_detector_map_tuned
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector_map_tuned
  runs_dir: runs/baseline_detector_map_tuned
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5

```

configs/base_tuned.yaml

```
experiment:
  name: baseline_detector_tuned
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector_tuned
  runs_dir: runs/baseline_detector_tuned
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/base.yaml

```
experiment:
  name: baseline_detector
  seed: 42
  output_dir: outputs/checkpoints/baseline_detector
  runs_dir: runs/baseline_detector
  device: cuda

dataset:
  train_root: data/datasets/URPC2018/train/images
  val_root: data/datasets/URPC2018/val/images
  enhanced_root: data/datasets/URPC2018_enhanced
  num_classes: 4
  synthetic_size: 16
  max_objects: 8
```

```
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 50
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 8
    min_delta: 0.0005

eval:
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_brackish_full.yaml

```
experiment:
  name: baseline_brackish_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_full
  runs_dir: runs/baseline_brackish_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
```

```
batch_size: 4
max_batches_per_epoch: 0
optimizer: adamw
lr: 0.0005
weight_decay: 0.0005
early_stopping:
  enabled: true
  monitor: loss
  mode: min
  patience: 10
  min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_brackish_rf_full.yaml

```
experiment:
  name: baseline_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_full
  runs_dir: runs/baseline_brackish_rf_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.25
iou_thresh: 0.5
```

configs/formal/base_brackish_rf_map_tuned_full.yaml

```
experiment:
  name: baseline_brackish_rf_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_map_tuned_full
  runs_dir: runs/baseline_brackish_rf_map_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/base_brackish_rf_tuned_full.yaml

```
experiment:
  name: baseline_brackish_rf_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_brackish_rf_tuned_full
  runs_dir: runs/baseline_brackish_rf_tuned_full
  device: cuda

dataset:
  name: brackish_rf
```

```
train_root: data/datasets/downloads/Brackish/train/images
val_root: data/datasets/downloads/Brackish/valid/images
enhanced_root: data/datasets/downloads/Brackish/train/images
num_classes: 6
synthetic_size: 0
max_objects: 16
class_names:
  - crab
  - fish
  - jellyfish
  - shrimp
  - small_fish
  - starfish
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/base_ruod_full.yaml

```
experiment:
  name: baseline_ruod_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_full
  runs_dir: runs/baseline_ruod_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: baseline_detector
```

```
feature_dim: 32
num_queries: 8
backbone: single_branch
head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_map_tuned_full.yaml

```
experiment:
  name: baseline_ruod_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_map_tuned_full
  runs_dir: runs/baseline_ruod_map_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
```

```
min_delta: 0.0005
```

```
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.01  
  iou_thresh: 0.5
```

configs/formal/base_ruod_testb_eval.yaml

```
experiment:  
  name: baseline_ruod_testb_eval  
  seed: 42  
  output_dir: outputs/checkpoints/baseline_ruod_full  
  runs_dir: runs/baseline_ruod_testb_eval  
  device: cuda  
  
dataset:  
  name: ruod_testb  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/testb/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images  
  num_classes: 4  
  synthetic_size: 0  
  max_objects: 8  
  class_names:  
    - holothurian  
    - echinus  
    - starfish  
    - scallop  
  image_size: 320  
  workers: 4  
  
model:  
  name: baseline_detector  
  feature_dim: 32  
  num_queries: 8  
  backbone: single_branch  
  head: detect_proxy  
  
train:  
  epochs: 100  
  batch_size: 4  
  max_batches_per_epoch: 0  
  optimizer: adamw  
  lr: 0.0005  
  weight_decay: 0.0005  
  
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.25  
  iou_thresh: 0.5
```

configs/formal/base_ruod_testc_eval.yaml

```
experiment:  
  name: baseline_ruod_testc_eval  
  seed: 42  
  output_dir: outputs/checkpoints/baseline_ruod_full  
  runs_dir: runs/baseline_ruod_testc_eval  
  device: cuda  
  
dataset:  
  name: ruod_testc  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/testc/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
```

```
num_classes: 4
synthetic_size: 0
max_objects: 8
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 320
workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_testl_eval.yaml

```
experiment:
  name: baseline_ruod_testl_eval
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_full
  runs_dir: runs/baseline_ruod_testl_eval
  device: cuda

dataset:
  name: ruod_testl
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testl/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
```

```
lr: 0.0005
weight_decay: 0.0005
```

```
eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_ruod_tuned_full.yaml

```
experiment:
  name: baseline_ruod_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_ruod_tuned_full
  runs_dir: runs/baseline_ruod_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4
  prefetch_factor: 2
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.05
iou_thresh: 0.5
```

configs/formal/base_uov2_full.yaml

```
experiment:
  name: baseline_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_full
  runs_dir: runs/baseline_uov2_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 8
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 320
  workers: 4

model:
  name: baseline_detector
  feature_dim: 32
  num_queries: 8
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/base_uov2_map_tuned_full.yaml

```
experiment:
  name: baseline_uov2_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_map_tuned_full
  runs_dir: runs/baseline_uov2_map_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
```

```
enhanced_root: data/datasets/downloads/U0v2/train/images
num_classes: 5
synthetic_size: 0
max_objects: 16
class_names:
  - echinus
  - holothurian
  - scallop
  - starfish
  - waterweeds
image_size: 640
workers: 4

model:
  name: baseline_detector
  feature_dim: 64
  num_queries: 16
  backbone: single_branch
  head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/base_uov2_tuned_full.yaml

```
experiment:
  name: baseline_uov2_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/baseline_uov2_tuned_full
  runs_dir: runs/baseline_uov2_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: baseline_detector
  feature_dim: 64
```

```
num_queries: 16
backbone: single_branch
head: detect_proxy

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_brackish_rf_full.yaml

```
experiment:
  name: faster_rcnn_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_brackish_rf_full
  runs_dir: runs/faster_rcnn_brackish_rf_full
  device: cuda
dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 32
  class_names: [crab, fish, jellyfish, shrimp, small_fish, starfish]
  image_size: 640
  workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_ruod_full.yaml

```
experiment:
  name: faster_rcnn_ruod_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_ruod_full
  runs_dir: runs/faster_rcnn_ruod_full
  device: cuda
dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
```

```
enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
num_classes: 4
synthetic_size: 0
max_objects: 32
class_names: [holothurian, echinus, starfish, scallop]
image_size: 640
workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/faster_rcnn_uov2_full.yaml

```
experiment:
  name: faster_rcnn_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/faster_rcnn_uov2_full
  runs_dir: runs/faster_rcnn_uov2_full
  device: cuda
dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 32
  class_names: [echinus, holothurian, scallop, starfish, waterweeds]
  image_size: 640
  workers: 4
model:
  name: faster_rcnn
train:
  epochs: 20
  batch_size: 2
  max_batches_per_epoch: 0
  lr: 0.005
  weight_decay: 0.0005
eval:
  split: val
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_full.yaml

```
experiment:
  name: qdcr_brackish_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_full
  runs_dir: runs/qdcr_brackish_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
```

```
class_names:
  - crab
  - fish
  - jellyfish
  - shrimp
  - small_fish
  - starfish
image_size: 320
workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_full.yaml

```
experiment:
  name: qdcr_brackish_rf_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_full
  runs_dir: runs/qdcr_brackish_rf_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 8
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 320
  workers: 4

model:
```

```
name: qdcr_net
feature_dim: 32
num_queries: 8
backbone: dual_branch
quality_aware: true
cross_residual_stages:
  - stage2
  - stage3
small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_map_tuned_full.yaml

```
experiment:
  name: qdcr_brackish_rf_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_map_tuned_full
  runs_dir: runs/qdcr_brackish_rf_map_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
```

```
epochs: 120
batch_size: 4
max_batches_per_epoch: 0
optimizer: adamw
lr: 0.0003
weight_decay: 0.0005
early_stopping:
  enabled: true
  monitor: map50
  mode: max
  patience: 20
  min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_brackish_rf_tuned_full.yaml

```
experiment:
  name: qdcr_brackish_rf_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_brackish_rf_tuned_full
  runs_dir: runs/qdcr_brackish_rf_tuned_full
  device: cuda

dataset:
  name: brackish_rf
  train_root: data/datasets/downloads/Brackish/train/images
  val_root: data/datasets/downloads/Brackish/valid/images
  enhanced_root: data/datasets/downloads/Brackish/train/images
  num_classes: 6
  synthetic_size: 0
  max_objects: 16
  class_names:
    - crab
    - fish
    - jellyfish
    - shrimp
    - small_fish
    - starfish
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
```

```
min_delta: 0.0005
```

```
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.05  
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_full.yaml

```
experiment:  
  name: qdcr_ruod_full  
  seed: 42  
  output_dir: outputs/checkpoints/qdcr_ruod_full  
  runs_dir: runs/qdcr_ruod_full  
  device: cuda  
  
dataset:  
  train_root: data/datasets/downloads/RUOD/RUOD/train/images  
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images  
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images  
  num_classes: 4  
  synthetic_size: 0  
  max_objects: 8  
  class_names:  
    - holothurian  
    - echinus  
    - starfish  
    - scallop  
  image_size: 320  
  workers: 4  
  
model:  
  name: qdcr_net  
  feature_dim: 32  
  num_queries: 8  
  backbone: dual_branch  
  quality_aware: true  
  cross_residual_stages:  
    - stage2  
    - stage3  
  small_object_neck: true  
  
train:  
  epochs: 100  
  batch_size: 4  
  max_batches_per_epoch: 0  
  optimizer: adamw  
  lr: 0.001  
  weight_decay: 0.0005  
  early_stopping:  
    enabled: true  
    monitor: loss  
    mode: min  
    patience: 10  
    min_delta: 0.0005  
  
eval:  
  checkpoint: best  
  max_batches: 0  
  conf_thresh: 0.25  
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_map_tuned_full.yaml

```
experiment:  
  name: qdcr_ruod_map_tuned_full  
  seed: 42
```

```
output_dir: outputs/checkpoints/qdcr_ruod_map_tuned_full
runs_dir: runs/qdcr_ruod_map_tuned_full
device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_sparse_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_sparse_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_sparse_tuned_full
  runs_dir: runs/qdcr_ruod_sparse_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
```

```
- holothurian
- echinus
- starfish
- scallop
image_size: 640
workers: 1
prefetch_factor: 1
cache_preprocessed: true
cache_dir: /tmp/qdcr_preprocessed/ruod_formal
```

```
model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_mode: sparse_residual
  cross_residual_threshold_init: 0.1
  cross_residual_threshold_slope: 10.0
  cross_residual_sparse_mode: soft_threshold
  cross_residual_sparse_lambda_init: 0.1
  cross_residual_topk_ratio: 0.5
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005
```

```
eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testb_eval.yaml

```
experiment:
  name: qdcr_ruod_testb_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testb_eval
  device: cuda

dataset:
  name: ruod_testb
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testb/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
```

```
max_objects: 8
class_names:
  - holothurian
  - echinus
  - starfish
  - scallop
image_size: 320
workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testc_eval.yaml

```
experiment:
  name: qdcr_ruod_testc_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testc_eval
  device: cuda

dataset:
  name: ruod_testc
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testc/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_testl_eval.yaml

```
experiment:
  name: qdcr_ruod_testl_eval
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_full
  runs_dir: runs/qdcr_ruod_testl_eval
  device: cuda

dataset:
  name: ruod_testl
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/testl/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 8
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.001
  weight_decay: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_ruod_threshold_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_threshold_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_threshold_tuned_full
  runs_dir: runs/qdcr_ruod_threshold_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 1
  prefetch_factor: 1
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_mode: learnable_threshold
  cross_residual_threshold_init: 0.1
  cross_residual_threshold_slope: 10.0
  cross_residual_sparse_mode: soft_threshold
  cross_residual_sparse_lambda_init: 0.1
  cross_residual_topk_ratio: 0.5
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
```

```
conf_thresh: 0.05
iou_thresh: 0.5
```

configs/formal/qdcr_ruod_tuned_full.yaml

```
experiment:
  name: qdcr_ruod_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_ruod_tuned_full
  runs_dir: runs/qdcr_ruod_tuned_full
  device: cuda

dataset:
  name: ruod
  train_root: data/datasets/downloads/RUOD/RUOD/train/images
  val_root: data/datasets/downloads/RUOD/RUOD/valid/images
  enhanced_root: data/datasets/downloads/RUOD/RUOD/train/images
  num_classes: 4
  synthetic_size: 0
  max_objects: 16
  class_names:
    - holothurian
    - echinus
    - starfish
    - scallop
  image_size: 640
  workers: 1
  prefetch_factor: 1
  cache_preprocessed: true
  cache_dir: /tmp/qdcr_preprocessed/ruod_formal

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 100
  max_batches_per_epoch: 0
  log_interval: 10
  scalar_log_interval: 10
  eval_interval: 5
  amp: true
  log_graph: false
  log_histograms: false
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.05
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_full.yaml

```
experiment:
  name: qdcr_uov2_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_full
  runs_dir: runs/qdcr_uov2_full
  device: cuda

dataset:
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 8
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 320
  workers: 4

model:
  name: qdcr_net
  feature_dim: 32
  num_queries: 8
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 100
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0005
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 10
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.25
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_probe_v2.yaml

```
experiment:
  name: qdcr_uov2_map_probe_v2
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_probe_v2
  runs_dir: runs/qdcr_uov2_map_probe_v2
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
```

```
enhanced_root: data/datasets/downloads/U0v2/train/images
num_classes: 5
synthetic_size: 0
max_objects: 16
class_names:
  - echinus
  - holothurian
  - scallop
  - starfish
  - waterweeds
image_size: 640
workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 20
  batch_size: 4
  max_batches_per_epoch: 120
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 8
    min_delta: 0.000001

eval:
  checkpoint: best
  max_batches: 80
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_probe.yaml

```
experiment:
  name: qdcr_uov2_map_probe
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_probe
  runs_dir: runs/qdcr_uov2_map_probe
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4
```

```
model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 12
  batch_size: 4
  max_batches_per_epoch: 120
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 6
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 60
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_map_tuned_full.yaml

```
experiment:
  name: qdcr_uov2_map_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_map_tuned_full
  runs_dir: runs/qdcr_uov2_map_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true
```

```
train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: map50
    mode: max
    patience: 20
    min_delta: 0.0005

eval:
  checkpoint: best
  max_batches: 0
  conf_thresh: 0.01
  iou_thresh: 0.5
```

configs/formal/qdcr_uov2_tuned_full.yaml

```
experiment:
  name: qdcr_uov2_tuned_full
  seed: 42
  output_dir: outputs/checkpoints/qdcr_uov2_tuned_full
  runs_dir: runs/qdcr_uov2_tuned_full
  device: cuda

dataset:
  name: uov2
  train_root: data/datasets/downloads/U0v2/train/images
  val_root: data/datasets/downloads/U0v2/valid/images
  enhanced_root: data/datasets/downloads/U0v2/train/images
  num_classes: 5
  synthetic_size: 0
  max_objects: 16
  class_names:
    - echinus
    - holothurian
    - scallop
    - starfish
    - waterweeds
  image_size: 640
  workers: 4

model:
  name: qdcr_net
  feature_dim: 64
  num_queries: 16
  backbone: dual_branch
  quality_aware: true
  cross_residual_stages:
    - stage2
    - stage3
  small_object_neck: true

train:
  epochs: 120
  batch_size: 4
  max_batches_per_epoch: 0
  optimizer: adamw
  lr: 0.0003
  weight_decay: 0.0005
  early_stopping:
    enabled: true
    monitor: loss
    mode: min
    patience: 15
```

min_delta: 0.0005

eval:

checkpoint: best

max_batches: 0

conf_thresh: 0.05

iou_thresh: 0.5